TRAMIA

Virtual Assets Infrastructure Platform

Technical Functional Specification

End-to-End Implementation Guide

Document Version	1.0
Date	January 2026
Classification	CONFIDENTIAL - Internal Use Only
Prepared By	Riverbank Solutions Limited
Status	Draft for Technical Review

A KCB Bank Group Innovation Lab Company

Table of Contents

Table of Contents....2
1. Introduction....5
1.1 Purpose....5
1.2 Scope....5
1.3 Target Audience....5
1.4 Definitions and Acronyms....5
2. System Architecture....7
2.1 Architecture Overview....7
2.2 Core Microservices....7
2.3 Blockchain Network Architecture....8
2.3.1 Network Configuration....8
2.3.2 Node Distribution....8
2.4 Integration Architecture....8
3. Wallet Infrastructure....10
3.1 Wallet Architecture Overview....10
3.1.1 Wallet Types....10
3.2 Wallet Provisioning Flow....10
3.3 Wallet Data Model....10
3.4 Balance Management....11
3.4.1 Balance Types....11
3.4.2 Balance Data Model....11
3.5 Supported Assets....12
3.5.1 Asset Registry....12
4. Core Banking Integration....13
4.1 Integration Architecture....13
4.1.1 Integration Components....13
4.2 Account Linking....13
4.3 Real-Time Balance Sync....13
4.3.1 Balance Aggregation API....14
4.4 Mobile App Integration (SDK)....14
4.4.1 SDK Initialization....14
5. On/Off Ramping System....16
5.1 Overview....16
5.2 On-Ramp (Fiat to Digital Assets)....16
5.2.1 Supported On-Ramp Methods....16

5.2.2 On-Ramp Flow (Bank Transfer - Instant) 16
5.2.3 On-Ramp API 16
5.3 Off-Ramp (Digital Assets to Fiat) 17
5.3.1 Off-Ramp Flow 17
5.3.2 Off-Ramp API 17
5.4 Liquidity Management 18
6. Trading Engine 19
6.1 Overview 19
6.2 Supported Trading Pairs 19
6.3 Order Types 19
6.4 Order Lifecycle 19
7. Real-World Asset Tokenization Engine 21
7.1 Overview 21
7.2 Tokenization Architecture 21
7.2.1 Smart Contract Stack 21
7.2.2 Tokenization Data Model 21
7.3 Generic Tokenization Workflow 22
7.4 Real Estate Tokenization 24
7.4.1 Overview 24
7.4.2 Required Documentation 24
7.4.3 Real Estate Data Model 24
7.4.4 Real Estate Token Economics 25
7.5 Invoice Tokenization 26
7.5.1 Overview 26
7.5.2 Invoice Tokenization Flow 26
7.5.3 Invoice Data Model 26
7.5.4 Invoice Risk Grading 27
7.6 Purchase Order (PO) Tokenization 28
7.6.1 Overview 28
7.6.2 PO Tokenization Flow 28
7.6.3 PO Data Model 28
7.7 Farm Produce Tokenization 30
7.7.1 Overview 30
7.7.2 Supported Agricultural Assets 30
7.7.3 Warehouse Receipt Integration 30
7.7.4 Agricultural Token Economics 32

7.8 Additional Tokenizable Asset Types 33
7.8.1 Government Bonds 33
7.8.2 Private Equity / SME Shares 33
7.8.3 Equipment / Vehicle Financing 33
7.8.4 Intellectual Property 33
8. Compliance Engine 34
8.1 Overview 34
8.2 KYC Tiers 34
8.3 Transaction Monitoring 34
8.4 Regulatory Reporting 34
9. API Specifications 35
9.1 API Design Principles 35
9.2 Core API Endpoints 35
9.2.1 Wallet APIs 35
9.2.2 Trading APIs 35
9.2.3 On/Off Ramp APIs 35
9.2.4 Tokenization APIs 35
9.3 Webhook Events 36
10. Security Architecture 37
10.1 Security Layers 37
10.2 Key Management Architecture 37
10.2.1 MPC Implementation (User Wallets) 37
10.2.2 Hot/Cold Wallet Strategy 37
11. Testing & Deployment 38
11.1 Testing Strategy 38
11.2 Environment Strategy 38
12. Appendix 38
12.1 Technology Stack Summary 38
12.2 Document Revision History 39

1. Introduction

1.1 Purpose

This Technical Functional Specification (TFS) document provides comprehensive guidance for the development team to build the Tramia platform end-to-end. It covers all functional requirements, technical architecture, data models, API specifications, integration patterns, and implementation details necessary to deliver a production-ready virtual assets infrastructure platform.

1.2 Scope

This document covers the following core capabilities:

- **Virtual Assets Infrastructure:** Complete blockchain infrastructure for digital asset custody, trading, and settlement
- **Banking-Integrated Wallet System:** Wallet provisioning for retail bank customers with seamless core banking integration
- **On/Off Ramping:** Fiat-to-crypto and crypto-to-fiat conversion with real-time settlement
- **Real-World Asset Tokenization:** End-to-end tokenization for Real Estate, Purchase Orders, Invoices, Farm Produce, and extensible asset types
- **Compliance Engine:** KYC/AML integration, transaction monitoring, and regulatory reporting
- **Administration Portal:** Operations dashboard, asset management, and system configuration

1.3 Target Audience

This document is intended for backend engineers, frontend developers, blockchain developers, DevOps engineers, QA engineers, solution architects, and technical project managers involved in the Tramia platform development.

1.4 Definitions and Acronyms

Term	Definition
VASP	Virtual Asset Service Provider
RWA	Real-World Asset
KYC	Know Your Customer
AML	Anti-Money Laundering
MPC	Multi-Party Computation (for key management)
HSM	Hardware Security Module
SPV	Special Purpose Vehicle
T24	Temenos T24 Core Banking System

ERC-3643	Token standard for compliant security tokens
CBDC	Central Bank Digital Currency

2. System Architecture

2.1 Architecture Overview

Tramia employs a layered microservices architecture designed for scalability, security, and regulatory compliance. The platform consists of four primary layers, each with distinct responsibilities and clear interfaces.

Layer	Components	Technology Stack
Presentation	Mobile Apps, Web Portal, Merchant Dashboard, Admin Console	React Native, React.js, Next.js
API Gateway	Authentication, Rate Limiting, Request Routing, API Versioning	Kong / AWS API Gateway
Application Services	Wallet Service, Trading Engine, Tokenization Service, Compliance Service	Node.js, Go, Python
Blockchain Layer	Smart Contracts, Validator Nodes, Event Listeners	Hyperledger Besu, Solidity
Data Layer	Relational DB, Document Store, Cache, Message Queue	PostgreSQL, MongoDB, Redis, RabbitMQ

2.2 Core Microservices

The platform is decomposed into the following microservices, each independently deployable and scalable:

Service	Responsibility
identity-service	User authentication, authorization, session management, OAuth2/OIDC provider
wallet-service	Wallet creation, address derivation, balance tracking, transaction history
custody-service	Key management (MPC/HSM), transaction signing, cold/hot wallet management
trading-service	Order matching, price feeds, liquidity management, settlement
fiat-gateway-service	On/off ramping, bank account linking, fiat transfers, reconciliation
tokenization-service	Asset onboarding, token minting, compliance rules, ownership registry
compliance-service	KYC verification, AML screening, transaction monitoring, sanctions checks
notification-service	Push notifications, SMS, email, in-app messaging
reporting-service	Analytics, regulatory reports, audit logs, dashboards
core-banking-bridge	T24 integration, account sync, real-time balance updates

blockchain-indexer	Event processing, transaction indexing, state synchronization
admin-service	System configuration, user management, operations dashboard

2.3 Blockchain Network Architecture

Tramia operates on a permissioned Hyperledger Besu network configured for enterprise use. The network provides EVM compatibility for smart contract deployment while enabling private transactions, permissioned access, and controlled gas costs.

2.3.1 Network Configuration

Parameter	Value
Consensus Mechanism	QBFT (Quorum Byzantine Fault Tolerance)
Block Time	2 seconds
Gas Limit per Block	30,000,000
Gas Price Strategy	Zero gas for whitelisted contracts; subsidized gas for users
Validator Nodes	4 initial validators (expandable to 7 for regional deployment)
Node Types	Validator nodes, Transaction nodes, Archive nodes
Privacy	Private transaction groups for sensitive RWA data

2.3.2 Node Distribution

- **Validator Node 1:** Primary data center (Nairobi)
- **Validator Node 2:** DR site (Mombasa or secondary cloud region)
- **Validator Node 3:** Partner institution (KCB infrastructure)
- **Validator Node 4:** Regulatory observer node (CMA/CBK optional participation)

2.4 Integration Architecture

The platform integrates with multiple external systems through well-defined interfaces:

System	Integration Type	Purpose
T24 Core Banking	REST API + MQ	Account data, balance sync, transaction posting
KCB Mobile App	SDK + API	Embedded wallet UI, seamless UX
IPRS/eCitizen	REST API	National ID verification for KYC
Credit Bureau	REST API	Credit scoring for invoice/PO financing
Land Registry	API (future)	Property ownership verification
Price Oracles	Chainlink/Custom	Real-time crypto and commodity prices

M-Pesa	Daraja API	Mobile money on/off ramp
SWIFT/RTGS	Bank channels	International and local fiat settlement

3. Wallet Infrastructure

3.1 Wallet Architecture Overview

Every retail bank customer receives a Tramia wallet automatically provisioned upon first access to digital asset features. The wallet system employs a hierarchical deterministic (HD) wallet structure with MPC-based key management to eliminate single points of failure while enabling seamless user experience.

3.1.1 Wallet Types

Wallet Type	Purpose	Key Management
User Wallet	Individual customer holdings	MPC (2-of-3: user device, server, backup)
Hot Wallet	Operational liquidity for trading	HSM-protected, multi-sig (3-of-5)
Cold Wallet	Long-term asset storage	Air-gapped HSM, multi-sig (4-of-7)
Escrow Wallet	Trade settlement, tokenization	Smart contract controlled
Fee Wallet	Gas subsidization, fee collection	Automated replenishment
Treasury Wallet	Platform reserves	Board-level multi-sig (5-of-9)

3.2 Wallet Provisioning Flow

The wallet provisioning process is triggered when a bank customer first accesses digital asset features:

1. **Trigger:** Customer opens 'Digital Assets' section in KCB mobile app
2. **KYC Verification:** System verifies customer has completed bank KYC (Level 2 or above)
3. **Terms Acceptance:** Customer reviews and accepts digital asset terms and conditions
4. **MPC Key Generation:** custody-service initiates distributed key generation ceremony
5. **Device Key Share:** Customer's device generates and stores local key share (encrypted with device biometrics)
6. **Server Key Share:** Server generates and stores HSM-protected key share
7. **Backup Key Share:** Encrypted backup share stored in secure vault (recoverable via KYC re-verification)
8. **Address Derivation:** HD wallet derives addresses for each supported chain/token
9. **Registration:** Wallet addresses registered in Identity Registry smart contract
10. **Activation:** Wallet activated and linked to customer's bank account

3.3 Wallet Data Model

```
-- Core wallet table
CREATE TABLE wallets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    customer_id UUID NOT NULL REFERENCES customers(id),
    bank_account_id VARCHAR(50) NOT NULL,
    wallet_type VARCHAR(20) NOT NULL DEFAULT 'USER',
    status VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    activated_at TIMESTAMP WITH TIME ZONE,
    kyc_level INTEGER NOT NULL DEFAULT 0,
    daily_limit_usd DECIMAL(18,2) DEFAULT 1000.00,
    monthly_limit_usd DECIMAL(18,2) DEFAULT 10000.00,
    metadata JSONB,

    CONSTRAINT wallet_type_check CHECK (
        wallet_type IN ('USER', 'HOT', 'COLD', 'ESCROW', 'FEE', 'TREASURY')
    ),
    CONSTRAINT status_check CHECK (
        status IN ('PENDING', 'ACTIVE', 'SUSPENDED', 'CLOSED')
    )
);

-- Wallet addresses per chain
CREATE TABLE wallet_addresses (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wallet_id UUID NOT NULL REFERENCES wallets(id),
    chain VARCHAR(20) NOT NULL,
    address VARCHAR(100) NOT NULL,
    derivation_path VARCHAR(100),
    is_primary BOOLEAN DEFAULT true,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(chain, address)
);

-- Supported chains: 'TRAMIA' (native), 'ETH', 'BTC', 'POLYGON'
```

3.4 Balance Management

3.4.1 Balance Types

Each wallet maintains multiple balance types for accurate tracking:

Balance Type	Description
available	Funds available for immediate use (trading, transfer, withdrawal)
pending	Funds in transit (deposits awaiting confirmation)
locked	Funds locked in open orders or pending transactions
staked	Funds committed to staking or yield programs
total	Sum of all balance types (available + pending + locked + staked)

3.4.2 Balance Data Model

```
CREATE TABLE wallet_balances (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    wallet_id UUID NOT NULL REFERENCES wallets(id),
    asset_id UUID NOT NULL REFERENCES assets(id),
    available DECIMAL(36,18) NOT NULL DEFAULT 0,
    pending DECIMAL(36,18) NOT NULL DEFAULT 0,
    locked DECIMAL(36,18) NOT NULL DEFAULT 0,
    staked DECIMAL(36,18) NOT NULL DEFAULT 0,
    last_updated TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    version INTEGER NOT NULL DEFAULT 1,

    UNIQUE(wallet_id, asset_id),
    CONSTRAINT positive_balances CHECK (
        available >= 0 AND pending >= 0 AND locked >= 0 AND staked >= 0
    )
);

-- Balance change audit log
CREATE TABLE balance_ledger (
    id BIGSERIAL PRIMARY KEY,
    wallet_id UUID NOT NULL,
    asset_id UUID NOT NULL,
    balance_type VARCHAR(20) NOT NULL,
    change_amount DECIMAL(36,18) NOT NULL,
    balance_before DECIMAL(36,18) NOT NULL,
    balance_after DECIMAL(36,18) NOT NULL,
    reference_type VARCHAR(50) NOT NULL,
    reference_id UUID NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    INDEX idx_balance_ledger_wallet (wallet_id, created_at DESC)
);
```

3.5 Supported Assets

3.5.1 Asset Registry

Asset	Symbol	Type	Contract/Chain
Bitcoin	**BTC**	Cryptocurrency	Native BTC (via custody bridge)
Ethereum	**ETH**	Cryptocurrency	Native ETH
USD Coin	**USDC**	Stablecoin	ERC-20 on Tramia chain
Tether	**USDT**	Stablecoin	ERC-20 on Tramia chain
Tramia KES	**tKES**	Fiat-backed	Native Tramia token (1:1 KES)
Tokenized RWA	**Various**	Security Token	ERC-3643 on Tramia chain

4. Core Banking Integration

4.1 Integration Architecture

The Core Banking Bridge (core-banking-bridge service) provides bidirectional integration between Tramia and the bank's T24 core banking system. This integration enables real-time account synchronization, seamless fund transfers, and unified customer experience.

4.1.1 Integration Components

Component	Direction	Protocol
Account Sync	T24 → Tramia	REST API (polling) + Event push
Balance Query	Tramia → T24	REST API (real-time)
Fund Transfer	Bidirectional	REST API + MQ confirmation
Transaction Post	Tramia → T24	Message Queue (guaranteed delivery)
Customer Data	T24 → Tramia	REST API (on-demand)
Statement Gen	Tramia → T24	Batch file (nightly)

4.2 Account Linking

When a customer activates their Tramia wallet, their bank account is automatically linked. The system supports multiple linked accounts per customer:

```
CREATE TABLE linked_bank_accounts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    wallet_id UUID NOT NULL REFERENCES wallets(id),
    bank_code VARCHAR(10) NOT NULL DEFAULT 'KCB',
    account_number VARCHAR(20) NOT NULL,
    account_type VARCHAR(20) NOT NULL,
    currency VARCHAR(3) NOT NULL DEFAULT 'KES',
    account_name VARCHAR(100),
    is_primary BOOLEAN DEFAULT false,
    is_verified BOOLEAN DEFAULT false,
    verified_at TIMESTAMP WITH TIME ZONE,
    status VARCHAR(20) DEFAULT 'ACTIVE',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(wallet_id, bank_code, account_number)
);

-- Verification requires micro-deposit confirmation or instant verification via T24
```

4.3 Real-Time Balance Sync

The customer's unified dashboard displays both fiat (KES) and digital asset balances. Fiat balances are fetched from T24 in real-time, while digital asset balances are maintained locally:

4.3.1 Balance Aggregation API

```
// GET /api/v1/wallets/{walletId}/balances/unified

{
  "wallet_id": "uuid",
  "customer_id": "uuid",
  "timestamp": "2026-01-15T10:30:00Z",
  "fiat_balances": [
    {
      "account_number": "1234567890",
      "account_type": "SAVINGS",
      "currency": "KES",
      "available": 150000.00,
      "current": 152000.00,
      "source": "T24"
    }
  ],
  "digital_balances": [
    {
      "asset": "BTC",
      "available": "0.05000000",
      "pending": "0.00000000",
      "locked": "0.00000000",
      "value_kes": 650000.00,
      "value_usd": 5000.00
    },
    {
      "asset": "tKES",
      "available": "25000.00000000",
      "pending": "0.00000000",
      "locked": "5000.00000000",
      "value_kes": 25000.00,
      "value_usd": 192.31
    }
  ],
  "total_value_kes": 827000.00,
  "total_value_usd": 6361.54
}
```

4.4 Mobile App Integration (SDK)

A dedicated SDK enables seamless integration of Tramia features into the KCB mobile banking app. The SDK provides:

- **Pre-built UI Components:** Wallet dashboard, asset cards, transaction history, trading interface
- **Secure Key Storage:** Biometric-protected local key share storage using device secure enclave
- **Transaction Signing:** Local signing flow with biometric confirmation
- **Push Notifications:** Real-time alerts for deposits, trades, and security events
- **Deep Linking:** Direct navigation from banking features to digital assets

4.4.1 SDK Initialization

```
// React Native SDK Integration

import { TramiaSDK, TramiaWallet, TramiaTrading } from '@tramia/mobile-sdk';

// Initialize SDK with bank authentication
await TramiaSDK.initialize({
  environment: 'production',
  bankCode: 'KCB',
  authToken: bankSessionToken,
  biometricRequired: true,
  onSessionExpiry: () => navigateToLogin()
});

// Get wallet instance
const wallet = await TramiaWallet.getInstance();

// Display wallet dashboard (pre-built UI)
<TramiaWalletDashboard
  onAssetPress={(asset) => navigateToAssetDetail(asset)}
  onSendPress={() => navigateToSend()}
  onReceivePress={() => navigateToReceive()}
  theme={bankTheme}
/>
```

5. On/Off Ramping System

5.1 Overview

The on/off ramping system enables seamless conversion between fiat currency (KES) and digital assets. By leveraging the direct banking integration, customers can fund their digital asset wallets and withdraw to their bank accounts instantly.

5.2 On-Ramp (Fiat to Digital Assets)

5.2.1 Supported On-Ramp Methods

Method	Speed	Limits	Fees
Bank Transfer (KCB)	Instant	KES 500 - 1M/day	0.5% (min KES 50)
Bank Transfer (Other)	1-2 hours	KES 1,000 - 500K/day	1.0% (min KES 100)
M-Pesa	Instant	KES 100 - 150K/day	1.5%
Card (Visa/MC)	Instant	KES 500 - 100K/tx	2.5%

5.2.2 On-Ramp Flow (Bank Transfer - Instant)

11. Customer initiates deposit from app (selects amount and source account)
12. System validates daily/monthly limits and compliance rules
13. fiat-gateway-service creates deposit order with unique reference
14. core-banking-bridge initiates debit from customer's KCB account
15. Upon T24 confirmation, system mints equivalent tKES tokens
16. tKES credited to customer's Tramia wallet (available balance)
17. Push notification sent to customer confirming deposit

5.2.3 On-Ramp API

```
// POST /api/v1/fiat/deposits

Request:
{
  "wallet_id": "uuid",
  "source_account_id": "uuid",
  "amount": 50000.00,
  "currency": "KES",
  "target_asset": "tKES",
  "idempotency_key": "unique-client-key"
}

Response:
{
  "deposit_id": "uuid",
```

```
  "status": "PROCESSING",
  "amount": 50000.00,
  "fee": 250.00,
  "net_amount": 49750.00,
  "target_asset": "tKES",
  "tokens_to_receive": "49750.000000000000000000",
  "exchange_rate": 1.0,
  "estimated_completion": "2026-01-15T10:30:05Z",
  "reference": "TRM-DEP-20260115-XXXXX"
}
```

5.3 Off-Ramp (Digital Assets to Fiat)

5.3.1 Off-Ramp Flow

18. Customer initiates withdrawal (selects asset, amount, destination account)
19. compliance-service runs AML checks on withdrawal request
20. If non-KES asset, trading-service executes market sell to tKES
21. tKES tokens burned from customer's wallet
22. fiat-gateway-service initiates bank transfer to destination account
23. For KCB accounts: instant credit via T24 internal transfer
24. For other banks: RTGS/EFT transfer (same-day or next-day)
25. Transaction recorded in both Tramia and T24 for reconciliation

5.3.2 Off-Ramp API

```
// POST /api/v1/fiat/withdrawals

Request:
{
  "wallet_id": "uuid",
  "destination_account_id": "uuid",
  "source_asset": "BTC",
  "amount": "0.01000000",
  "target_currency": "KES",
  "idempotency_key": "unique-client-key"
}

Response:
{
  "withdrawal_id": "uuid",
  "status": "PENDING_CONVERSION",
  "source_asset": "BTC",
  "source_amount": "0.01000000",
  "conversion_rate": 13000000.00,
  "gross_kes": 130000.00,
  "fees": {
    "conversion_fee": 1300.00,
    "withdrawal_fee": 650.00,
    "total_fees": 1950.00
  },
  "net_kes": 128050.00,
  "destination_account": "****7890",
```

```
  "estimated_arrival": "2026-01-15T11:00:00Z",
  "reference": "TRM-WDR-20260115-XXXXX"
}
```

5.4 Liquidity Management

The platform maintains liquidity pools to ensure instant on/off ramping. The Treasury Management module monitors reserves and triggers automated rebalancing:

Pool	Target Balance	Min Threshold	Rebalance Action
KES Float (T24)	KES 50M	KES 10M	Transfer from Treasury
tKES Reserve	Match Float	1:1 with KES	Mint/Burn to match
BTC Hot Wallet	5 BTC	1 BTC	Move from Cold
ETH Hot Wallet	50 ETH	10 ETH	Move from Cold
USDC Reserve	$500K	$100K	Purchase from LP

6. Trading Engine

6.1 Overview

The trading engine supports both cryptocurrency trading (BTC, ETH, stablecoins) and tokenized real-world asset trading. It implements a hybrid model combining an order book for liquid pairs and AMM (Automated Market Maker) pools for less liquid tokenized assets.

6.2 Supported Trading Pairs

Pair	Type	Mechanism	Min Order
BTC/tKES	Crypto/Fiat	Order Book	KES 500
ETH/tKES	Crypto/Fiat	Order Book	KES 500
BTC/USDC	Crypto/Stable	Order Book	$10
ETH/USDC	Crypto/Stable	Order Book	$10
USDC/tKES	Stable/Fiat	Fixed Rate + Spread	KES 100
RWA/tKES	Token/Fiat	AMM Pool	KES 100

6.3 Order Types

- **Market Order:** Execute immediately at best available price
- **Limit Order:** Execute at specified price or better
- **Stop-Loss Order:** Trigger market sell when price drops to threshold
- **Recurring Buy:** Automated periodic purchases (DCA strategy)

6.4 Order Lifecycle

```
Order States:
PENDING → VALIDATING → OPEN → PARTIALLY_FILLED → FILLED
                  ↓           ↓
               REJECTED    CANCELLED

// POST /api/v1/trading/orders

Request:
{
  "wallet_id": "uuid",
  "pair": "BTC/tKES",
  "side": "BUY",
  "type": "LIMIT",
  "amount": "0.01000000",
  "price": "13000000.00",
  "time_in_force": "GTC",
  "client_order_id": "client-unique-id"
}

Response:
```

```
{
  "order_id": "uuid",
  "status": "OPEN",
  "pair": "BTC/tKES",
  "side": "BUY",
  "type": "LIMIT",
  "amount": "0.01000000",
  "filled_amount": "0.00000000",
  "remaining": "0.01000000",
  "price": "13000000.00",
  "average_fill_price": null,
  "total_cost": "130000.00",
  "fees_estimated": "1300.00",
  "created_at": "2026-01-15T10:30:00Z"
}
```

7. Real-World Asset Tokenization Engine

7.1 Overview

The Tokenization Engine is the core capability that transforms real-world assets into blockchain-based tokens. It handles the entire lifecycle from asset onboarding through token minting, trading, distributions, and eventual redemption. The engine implements ERC-3643 compliant tokens with built-in identity verification and transfer restrictions.

7.2 Tokenization Architecture

7.2.1 Smart Contract Stack

Contract	Purpose
IdentityRegistry	Maps wallet addresses to verified identities; stores KYC claim hashes
ClaimTopicsRegistry	Defines required claims for each asset class (accreditation, jurisdiction, etc.)
TrustedIssuersRegistry	Whitelists identity verifiers (KCB, third-party KYC providers)
ComplianceContract	Enforces transfer rules (investor limits, lock-ups, jurisdiction restrictions)
AssetFactory	Deploys new token contracts with standardized configuration
TokenContract	ERC-3643 token with transfer hooks to compliance module
DividendDistributor	Manages yield/dividend payments to token holders
Marketplace	Secondary trading with compliance-checked order matching

7.2.2 Tokenization Data Model

```
-- Asset being tokenized
CREATE TABLE tokenized_assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_type VARCHAR(50) NOT NULL,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    issuer_id UUID NOT NULL REFERENCES issuers(id),
    legal_entity_id UUID REFERENCES legal_entities(id),

    -- Token details
    token_symbol VARCHAR(20) NOT NULL UNIQUE,
    token_name VARCHAR(100) NOT NULL,
    token_contract_address VARCHAR(100),
    total_supply DECIMAL(36,18),
    tokens_outstanding DECIMAL(36,18),
    token_price_kes DECIMAL(18,4),

    -- Asset valuation
    asset_value_kes DECIMAL(18,2),
    valuation_date DATE,
    valuation_provider VARCHAR(100),
```

```
    -- Offering details
    min_investment_kes DECIMAL(18,2) DEFAULT 100.00,
    max_investors INTEGER,
    current_investors INTEGER DEFAULT 0,
    offering_start DATE,
    offering_end DATE,

    -- Status
    status VARCHAR(30) NOT NULL DEFAULT 'DRAFT',
    compliance_status VARCHAR(30) DEFAULT 'PENDING',

    -- Metadata
    documents JSONB,
    metadata JSONB,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    CONSTRAINT asset_type_check CHECK (
        asset_type IN ('REAL_ESTATE', 'INVOICE', 'PURCHASE_ORDER',
                       'FARM_PRODUCE', 'BOND', 'EQUITY', 'COMMODITY')
    ),
    CONSTRAINT status_check CHECK (
        status IN ('DRAFT', 'PENDING_APPROVAL', 'APPROVED', 'MINTING',
                   'ACTIVE', 'PAUSED', 'MATURED', 'REDEEMED', 'CANCELLED')
    )
);

-- Token holders registry (mirrors on-chain state)
CREATE TABLE token_holdings (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    asset_id UUID NOT NULL REFERENCES tokenized_assets(id),
    wallet_id UUID NOT NULL REFERENCES wallets(id),
    balance DECIMAL(36,18) NOT NULL DEFAULT 0,
    locked_balance DECIMAL(36,18) NOT NULL DEFAULT 0,
    acquisition_date DATE,
    acquisition_price_avg DECIMAL(18,4),
    last_updated TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    UNIQUE(asset_id, wallet_id)
);
```

7.3 Generic Tokenization Workflow

All asset types follow a standardized workflow with asset-specific validation steps:

26. **Asset Submission:** Issuer submits asset details and supporting documents via portal
27. **Document Verification:** Legal team reviews ownership proof, valuations, encumbrances
28. **Legal Structuring:** Asset placed in SPV or trust structure if required
29. **Compliance Review:** CMA approval for securities; internal compliance sign-off
30. **Token Configuration:** Define token parameters (supply, restrictions, distribution rules)
31. **Smart Contract Deployment:** AssetFactory deploys new token contract with compliance rules
32. **Token Minting:** Initial tokens minted to issuer's escrow wallet

33. **Primary Offering:** Tokens sold to investors via platform (subscription period)
34. **Secondary Trading:** After lock-up period, tokens tradeable on marketplace
35. **Lifecycle Events:** Dividends, redemptions, corporate actions processed on-chain

7.4 Real Estate Tokenization

7.4.1 Overview

Real estate tokenization enables fractional ownership of property assets, allowing retail investors to participate in real estate investment with amounts as low as KES 100. Properties are held in SPVs, with tokens representing beneficial ownership shares.

7.4.2 Required Documentation

- Title Deed / Certificate of Lease
- Land Search Certificate (not older than 14 days)
- Property Valuation Report (from registered valuer)
- SPV Registration Documents (CR12, Memorandum)
- Building Plans and Approvals (for developments)
- Rental/Lease Agreements (for income-generating properties)
- Insurance Certificate

7.4.3 Real Estate Data Model

```
CREATE TABLE real_estate_assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tokenized_asset_id UUID NOT NULL REFERENCES tokenized_assets(id),

    -- Property details
    property_type VARCHAR(50) NOT NULL,
    address_line1 VARCHAR(200),
    address_line2 VARCHAR(200),
    city VARCHAR(100),
    county VARCHAR(100),
    country VARCHAR(3) DEFAULT 'KEN',
    coordinates POINT,

    -- Legal details
    title_number VARCHAR(100),
    lr_number VARCHAR(100),
    title_type VARCHAR(50),
    land_size_sqm DECIMAL(12,2),
    building_size_sqm DECIMAL(12,2),

    -- Valuation
    purchase_price_kes DECIMAL(18,2),
    current_valuation_kes DECIMAL(18,2),
    valuation_date DATE,
    valuer_name VARCHAR(200),
    valuer_registration VARCHAR(50),

    -- Income (if applicable)
    rental_income_monthly_kes DECIMAL(18,2),
    occupancy_rate DECIMAL(5,2),
    lease_expiry_date DATE,

    -- SPV details
    spv_name VARCHAR(200),
    spv_registration_number VARCHAR(50),
```

```
    spv_incorporation_date DATE,

    -- Documents (stored in secure document store)
    title_deed_doc_id UUID,
    valuation_report_doc_id UUID,
    spv_docs_id UUID,

    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    CONSTRAINT property_type_check CHECK (
        property_type IN ('RESIDENTIAL', 'COMMERCIAL', 'INDUSTRIAL',
                          'LAND', 'MIXED_USE', 'AGRICULTURAL')
    )
);
```

7.4.4 Real Estate Token Economics

Parameter	Value
Minimum Investment	KES 100
Token Denomination	KES 100 per token (configurable per asset)
Maximum Investors	Unlimited for public offerings; capped for private
Lock-up Period	6 months from purchase (configurable)
Distribution Frequency	Quarterly (rental income after expenses)
Management Fee	1-2% annual on NAV
Exit Mechanism	Secondary market sale or property sale proceeds distribution

7.5 Invoice Tokenization

7.5.1 Overview

Invoice tokenization enables businesses to convert their accounts receivable into tradeable tokens, providing immediate liquidity. Investors purchase invoice tokens at a discount and receive face value upon invoice payment. This creates a peer-to-peer invoice financing marketplace.

7.5.2 Invoice Tokenization Flow

36. Seller uploads invoice with supporting documents
37. System validates invoice authenticity (optional: buyer confirmation)
38. Credit scoring of buyer (debtor) via Credit Bureau integration
39. System calculates discount rate based on buyer risk and tenor
40. Invoice token minted and listed for investment
41. Investors purchase tokens (fund the invoice)
42. Seller receives immediate funds (minus discount and fees)
43. At maturity, buyer pays invoice to escrow account
44. Investors receive principal + yield; tokens burned

7.5.3 Invoice Data Model

```
CREATE TABLE invoice_assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tokenized_asset_id UUID NOT NULL REFERENCES tokenized_assets(id),

    -- Parties
    seller_id UUID NOT NULL REFERENCES business_customers(id),
    buyer_name VARCHAR(200) NOT NULL,
    buyer_registration VARCHAR(50),
    buyer_contact VARCHAR(100),

    -- Invoice details
    invoice_number VARCHAR(100) NOT NULL,
    invoice_date DATE NOT NULL,
    due_date DATE NOT NULL,
    face_value_kes DECIMAL(18,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'KES',

    -- Financing terms
    discount_rate DECIMAL(8,4) NOT NULL,
    advance_rate DECIMAL(5,2) DEFAULT 90.00,
    funded_amount_kes DECIMAL(18,2),

    -- Risk assessment
    buyer_credit_score INTEGER,
    buyer_payment_history JSONB,
    risk_grade VARCHAR(5),

    -- Status
    payment_status VARCHAR(30) DEFAULT 'PENDING',
    payment_date DATE,
    payment_amount_kes DECIMAL(18,2),
```

```
    days_past_due INTEGER DEFAULT 0,

    -- Verification
    buyer_confirmed BOOLEAN DEFAULT false,
    buyer_confirmed_at TIMESTAMP WITH TIME ZONE,

    -- Documents
    invoice_doc_id UUID,
    delivery_note_doc_id UUID,
    contract_doc_id UUID,

    CONSTRAINT payment_status_check CHECK (
        payment_status IN ('PENDING', 'PARTIAL', 'PAID', 'OVERDUE', 'DEFAULT')
    )
);
```

7.5.4 Invoice Risk Grading

Grade	Buyer Profile	Discount Rate (30d)	Max Advance
A+	Listed Corp / Government	1.5% - 2.0%	95%
A	Large Private Company	2.0% - 3.0%	90%
B+	Established SME	3.0% - 4.5%	85%
B	SME with Good History	4.5% - 6.0%	80%
C	New/Unrated Buyer	6.0% - 8.0%	70%

7.6 Purchase Order (PO) Tokenization

7.6.1 Overview

PO tokenization enables suppliers to obtain working capital against confirmed purchase orders before fulfillment. This is higher risk than invoice financing (since goods/services haven't been delivered yet), but provides crucial pre-shipment financing for suppliers.

7.6.2 PO Tokenization Flow

45. Supplier submits PO from verified buyer
46. System verifies PO authenticity with buyer (mandatory for PO financing)
47. Supplier capability assessment (can they fulfill the order?)
48. Risk scoring based on buyer, supplier, and transaction
49. PO token minted with milestone-based disbursement structure
50. Initial tranche released to supplier for procurement/production
51. Subsequent tranches released upon milestone verification
52. Upon delivery confirmation, PO converts to invoice
53. Buyer payment settles investor returns; tokens burned

7.6.3 PO Data Model

```
CREATE TABLE purchase_order_assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tokenized_asset_id UUID NOT NULL REFERENCES tokenized_assets(id),

    -- Parties
    supplier_id UUID NOT NULL REFERENCES business_customers(id),
    buyer_name VARCHAR(200) NOT NULL,
    buyer_registration VARCHAR(50),
    buyer_verified BOOLEAN DEFAULT false,
    buyer_verified_at TIMESTAMP WITH TIME ZONE,
    buyer_verification_ref VARCHAR(100),

    -- PO details
    po_number VARCHAR(100) NOT NULL,
    po_date DATE NOT NULL,
    delivery_deadline DATE NOT NULL,
    po_value_kes DECIMAL(18,2) NOT NULL,
    currency VARCHAR(3) DEFAULT 'KES',

    -- Goods/Services
    description TEXT,
    category VARCHAR(100),
    quantity DECIMAL(18,4),
    unit_of_measure VARCHAR(50),

    -- Financing structure
    max_advance_rate DECIMAL(5,2) DEFAULT 70.00,
    interest_rate_monthly DECIMAL(8,4),
    total_financing_kes DECIMAL(18,2),

    -- Milestones
    milestones JSONB,
```

```
    current_milestone INTEGER DEFAULT 0,

    -- Status
    fulfillment_status VARCHAR(30) DEFAULT 'PENDING',
    delivery_confirmed BOOLEAN DEFAULT false,
    delivery_date DATE,

    -- Risk
    supplier_score INTEGER,
    buyer_score INTEGER,
    overall_risk_grade VARCHAR(5),

    -- Documents
    po_doc_id UUID,
    contract_doc_id UUID,
    delivery_proof_doc_id UUID,

    CONSTRAINT fulfillment_status_check CHECK (
        fulfillment_status IN ('PENDING', 'IN_PROGRESS', 'SHIPPED',
                               'DELIVERED', 'ACCEPTED', 'DISPUTED')
    )
);

-- Milestone tracking
CREATE TABLE po_milestones (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    po_asset_id UUID NOT NULL REFERENCES purchase_order_assets(id),
    milestone_number INTEGER NOT NULL,
    description VARCHAR(200),
    target_date DATE,
    disbursement_percentage DECIMAL(5,2),
    disbursement_amount_kes DECIMAL(18,2),

    status VARCHAR(20) DEFAULT 'PENDING',
    completed_date DATE,
    verification_doc_id UUID,
    verified_by UUID,

    CONSTRAINT status_check CHECK (
        status IN ('PENDING', 'IN_PROGRESS', 'COMPLETED', 'VERIFIED', 'FAILED')
    )
);
```

7.7 Farm Produce Tokenization

7.7.1 Overview

Agricultural tokenization enables farmers and aggregators to tokenize crops (either pre-harvest or post-harvest), livestock, and agricultural commodities. This provides farmers with advance financing and gives investors exposure to agricultural returns. The system integrates with warehouse receipt systems for post-harvest tokenization.

7.7.2 Supported Agricultural Assets

Category	Examples	Tokenization Model
Pre-Harvest Crops	Maize, wheat, tea, coffee, horticulture	Forward contract tokenization
Warehouse Receipts	Stored grain, coffee, pulses	Receipt-backed tokens
Livestock	Cattle, sheep, goats, poultry	Asset-backed tokens
Dairy Production	Milk supply contracts	Receivables tokenization
Farmland	Agricultural land parcels	Real estate tokenization model

7.7.3 Warehouse Receipt Integration

For post-harvest tokenization, Tramia integrates with licensed warehouse operators to verify commodity deposits:

```
CREATE TABLE agricultural_assets (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tokenized_asset_id UUID NOT NULL REFERENCES tokenized_assets(id),

    -- Farmer/Aggregator
    farmer_id UUID REFERENCES farmers(id),
    aggregator_id UUID REFERENCES aggregators(id),
    cooperative_id UUID REFERENCES cooperatives(id),

    -- Product details
    commodity_type VARCHAR(100) NOT NULL,
    variety VARCHAR(100),
    grade VARCHAR(20),
    quantity DECIMAL(18,4) NOT NULL,
    unit VARCHAR(20) NOT NULL,

    -- Location
    farm_location_county VARCHAR(100),
    farm_location_subcounty VARCHAR(100),
    farm_gps_coordinates POINT,
    farm_size_acres DECIMAL(10,2),

    -- Timing
    planting_date DATE,
    expected_harvest_date DATE,
    actual_harvest_date DATE,

    -- Warehouse (for stored commodities)
```

```
    warehouse_id UUID REFERENCES warehouses(id),
    warehouse_receipt_number VARCHAR(100),
    warehouse_receipt_date DATE,
    storage_start_date DATE,
    storage_end_date DATE,

    -- Valuation
    spot_price_per_unit DECIMAL(18,4),
    forward_price_per_unit DECIMAL(18,4),
    total_value_kes DECIMAL(18,2),
    valuation_date DATE,

    -- Quality
    quality_grade VARCHAR(10),
    moisture_content DECIMAL(5,2),
    quality_certificate_id UUID,

    -- Insurance
    crop_insurance_policy VARCHAR(100),
    insurance_provider VARCHAR(100),
    insured_value_kes DECIMAL(18,2),

    -- Status
    harvest_status VARCHAR(30),
    delivery_status VARCHAR(30),

    CONSTRAINT commodity_type_check CHECK (
        commodity_type IN ('MAIZE', 'WHEAT', 'RICE', 'COFFEE', 'TEA',
                           'BEANS', 'SORGHUM', 'COTTON', 'SUGARCANE',
                           'CATTLE', 'SHEEP', 'GOATS', 'POULTRY', 'OTHER')
    )
);

-- Warehouse integration
CREATE TABLE warehouse_receipts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    agricultural_asset_id UUID REFERENCES agricultural_assets(id),
    warehouse_id UUID NOT NULL REFERENCES warehouses(id),

    receipt_number VARCHAR(100) NOT NULL UNIQUE,
    issue_date DATE NOT NULL,
    expiry_date DATE,

    commodity VARCHAR(100),
    quantity DECIMAL(18,4),
    unit VARCHAR(20),
    grade VARCHAR(20),

    depositor_name VARCHAR(200),
    depositor_id VARCHAR(50),

    status VARCHAR(20) DEFAULT 'ACTIVE',
    pledged BOOLEAN DEFAULT false,
    pledged_to UUID,

    verified_on_chain BOOLEAN DEFAULT false,
    blockchain_tx_hash VARCHAR(100)
);
```

7.7.4 Agricultural Token Economics

Parameter	Pre-Harvest	Warehouse Receipt
Advance Rate	50-60% of expected value	70-80% of spot value
Interest Rate	18-24% p.a.	14-18% p.a.
Tenor	Aligned to harvest (3-6 months)	1-6 months
Collateral	Standing crop + insurance	Warehouse receipt
Settlement	Sale proceeds at harvest	Commodity sale or physical delivery

7.8 Additional Tokenizable Asset Types

7.8.1 Government Bonds

Treasury bonds and infrastructure bonds can be tokenized to enable fractional investment. Integration with the Central Depository System (CDSC) enables automated settlement.

- **Minimum Investment:** KES 100 (vs. KES 50,000 direct)
- **Settlement:** T+0 on-platform; CDSC reconciliation T+1
- **Yield Distribution:** Automatic coupon distribution to token holders

7.8.2 Private Equity / SME Shares

Private company shares can be tokenized for cap table management and compliant secondary trading among qualified investors.

- **Investor Restrictions:** Accredited investors only (configurable per offering)
- **Transfer Restrictions:** Board approval, ROFR, lock-up periods enforced on-chain
- **Corporate Actions:** Dividends, stock splits, voting via smart contracts

7.8.3 Equipment / Vehicle Financing

High-value equipment (machinery, vehicles, medical equipment) can be tokenized for fractional financing:

- **Collateral:** Equipment title/logbook held in escrow
- **Valuation:** Depreciation-adjusted market value
- **Yield:** Lease/rental income distributed to token holders

7.8.4 Intellectual Property

Future capability for tokenizing music royalties, patents, and other IP with revenue-share tokens.

8. Compliance Engine

8.1 Overview

The Compliance Engine ensures all platform activities meet regulatory requirements under the VASP Act 2025, AML/CFT regulations, and securities laws. It operates as a real-time policy enforcement layer across all transactions.

8.2 KYC Tiers

Tier	Requirements	Daily Limit	Features
Tier 0	Bank KYC inherited	KES 10,000	View only, small deposits
Tier 1	+ ID verification	KES 100,000	Trading, basic tokenized assets
Tier 2	+ Address proof, source of funds	KES 1,000,000	Full trading, RWA investment
Tier 3	+ Enhanced due diligence	Unlimited	Institutional features, issuance

8.3 Transaction Monitoring

Real-time monitoring rules detect suspicious patterns:

- **Velocity Checks:** Unusual transaction frequency or volume
- **Structuring Detection:** Multiple transactions just below reporting thresholds
- **Sanctions Screening:** Real-time checks against OFAC, UN, and local sanctions lists
- **PEP Screening:** Politically Exposed Persons identification
- **Blockchain Analytics:** Risk scoring of external wallet addresses (Chainalysis integration)
- **Travel Rule:** FATF-compliant originator/beneficiary data sharing

8.4 Regulatory Reporting

Report	Frequency	Recipient
STR (Suspicious Transaction)	Within 24 hours	FRC (Financial Reporting Centre)
CTR (Currency Transaction)	Within 7 days	FRC
VASP Activity Report	Monthly	CMA
Tokenization Report	Quarterly	CMA
AML Compliance Report	Annually	CBK, FRC

9. API Specifications

9.1 API Design Principles

- REST with JSON payloads for synchronous operations
- WebSocket for real-time updates (prices, order status, notifications)
- GraphQL for flexible data queries (reporting, analytics)
- OAuth 2.0 + JWT for authentication
- API versioning via URL path (/api/v1/, /api/v2/)
- Idempotency keys for all state-changing operations

9.2 Core API Endpoints

9.2.1 Wallet APIs

```
POST   /api/v1/wallets                      Create new wallet
GET    /api/v1/wallets/{id}                   Get wallet details
GET    /api/v1/wallets/{id}/balances          Get all balances
GET    /api/v1/wallets/{id}/balances/unified  Get unified (fiat + crypto)
GET    /api/v1/wallets/{id}/transactions      Get transaction history
POST   /api/v1/wallets/{id}/addresses         Generate new deposit address
GET    /api/v1/wallets/{id}/addresses         List all addresses
```

9.2.2 Trading APIs

```
GET    /api/v1/trading/pairs                 List trading pairs
GET    /api/v1/trading/pairs/{pair}/orderbook Get order book
GET    /api/v1/trading/pairs/{pair}/ticker    Get current price
POST   /api/v1/trading/orders                 Create order
GET    /api/v1/trading/orders/{id}            Get order status
DELETE /api/v1/trading/orders/{id}            Cancel order
GET    /api/v1/trading/orders                 List user orders
POST   /api/v1/trading/swap                   Instant swap (market order)
```

9.2.3 On/Off Ramp APIs

```
POST   /api/v1/fiat/deposits                 Create deposit
GET    /api/v1/fiat/deposits/{id}             Get deposit status
POST   /api/v1/fiat/withdrawals               Create withdrawal
GET    /api/v1/fiat/withdrawals/{id}          Get withdrawal status
GET    /api/v1/fiat/bank-accounts             List linked accounts
POST   /api/v1/fiat/bank-accounts             Link new account
```

9.2.4 Tokenization APIs

```
POST   /api/v1/tokenization/assets           Submit asset for tokenization
GET    /api/v1/tokenization/assets/{id}       Get asset details
PUT    /api/v1/tokenization/assets/{id}       Update asset (draft status)
POST   /api/v1/tokenization/assets/{id}/submit Submit for approval
```

```
GET    /api/v1/tokenization/offerings         List active offerings
POST   /api/v1/tokenization/offerings/{id}/invest  Invest in offering
GET    /api/v1/tokenization/holdings          Get user's token holdings
GET    /api/v1/tokenization/marketplace       List secondary market
```

9.3 Webhook Events

Partner systems can subscribe to real-time events via webhooks:

Event	Trigger
wallet.created	New wallet provisioned
deposit.completed	Fiat deposit confirmed
withdrawal.completed	Withdrawal settled to bank
order.filled	Trade order fully executed
token.transfer	Token transferred between wallets
dividend.distributed	RWA dividend paid to holders
compliance.alert	Transaction flagged for review

10. Security Architecture

10.1 Security Layers

Layer	Controls
Network	WAF, DDoS protection, VPN for internal traffic, network segmentation
Application	Input validation, OWASP Top 10 mitigations, rate limiting
Authentication	MFA, biometric, session management, device binding
Authorization	RBAC, attribute-based policies, principle of least privilege
Data	Encryption at rest (AES-256), in transit (TLS 1.3), field-level encryption for PII
Key Management	HSM for hot keys, MPC for user keys, air-gapped cold storage
Blockchain	Permissioned network, validator authentication, smart contract audits

10.2 Key Management Architecture

10.2.1 MPC Implementation (User Wallets)

User keys are never reconstructed in a single location. Transaction signing uses threshold signatures with distributed key shares:

- **Share 1 (User Device):** Encrypted with biometric, stored in secure enclave
- **Share 2 (Tramia Server):** HSM-protected, requires authenticated session
- **Share 3 (Backup):** Encrypted, stored offline, recoverable via KYC re-verification

10.2.2 Hot/Cold Wallet Strategy

Wallet	Balance Policy	Access Control
Hot	Max 5% of AUM per asset	Automated within limits; manual approval above
Warm	15% of AUM (quick access reserve)	2-of-3 multi-sig, 1-hour delay
Cold	80% of AUM	4-of-7 multi-sig, air-gapped, 24hr delay

11. Testing & Deployment

11.1 Testing Strategy

Test Type	Scope
Unit Tests	Individual functions, smart contract methods; 80%+ coverage target
Integration Tests	Service-to-service communication, database interactions
E2E Tests	Complete user flows (wallet creation → trading → withdrawal)
Smart Contract Audit	Third-party security audit before mainnet deployment
Penetration Testing	Annual external pentest + continuous bug bounty
Load Testing	1000 TPS target, 10x burst capacity
Chaos Engineering	Failure injection to verify resilience

11.2 Environment Strategy

Environment	Purpose
Development	Local development with mock services, local blockchain node
Integration	Shared environment for testing service interactions
Staging	Production mirror with testnet blockchain, T24 sandbox
UAT	User acceptance testing with synthetic data
Production	Live environment with real assets and banking integration

12. Appendix

12.1 Technology Stack Summary

Component	Technology
Backend Services	Node.js (TypeScript), Go, Python
API Framework	NestJS, FastAPI, Gin
Blockchain	Hyperledger Besu (private), Ethereum (public bridge)
Smart Contracts	Solidity, OpenZeppelin, ERC-3643
Database	PostgreSQL 15, MongoDB 7, Redis 7
Message Queue	RabbitMQ, Apache Kafka (for event streaming)
Mobile SDK	React Native, Swift (iOS), Kotlin (Android)

Web Frontend	React.js, Next.js, TailwindCSS
Infrastructure	Kubernetes, Docker, Terraform
Cloud Provider	AWS (primary), Azure (DR)
Monitoring	Prometheus, Grafana, ELK Stack, PagerDuty

12.2 Document Revision History

Version	Date	Author	Changes
1.0	January 2026	Riverbank Solutions	Initial release

END OF DOCUMENT

This document is confidential and proprietary to Riverbank Solutions Limited.